Question 1: Web Server Reconnaissance and Vulnerability Analysis

A cybersecurity analyst is assigned an authorized and isolated vulnerable web server for asecurity assessment. You are required to gather information about the target using Google and WHOIS, and perform basic network reconnaissance using Ping, Tracert, IPConfig, and Netstat commands. Use Nmap to identify open ports, running services, and service versions. Use WhatWeb to identify the web technologies and frameworks used by the server. Examine HTTP response headers using Curl/Wget and identify information disclosure. Perform CGI and web-server vulnerability scanning using Nikto, and use Gobuster/Dirb to discover hidden directoriesand files. Analyze the identified administrative pages, backup files, configuration files, and vulnerable resources. Finally, classify the findings based on severity and impact, and recommend appropriate web-server hardening and information-disclosure prevention measures. Submit the commands executed, screenshots, observations, findings, and recommendations.

Aim

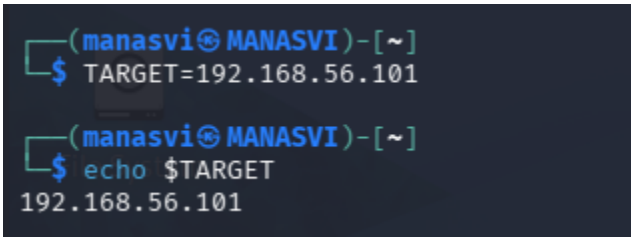
To perform web server reconnaissance and vulnerability analysis on an authorized vulnerable web server using Kali Linux tools. The assessment aims to identify network connectivity, open ports, running services, service versions, web technologies, HTTP information disclosure, vulnerabilities, and hidden directories/files, and to analyze the security risks associated with the findings.

1. Set the target

TARGET=192.168.56.101

Then verify:

echo $TARGET



2. Ping – Check Target Connectivity

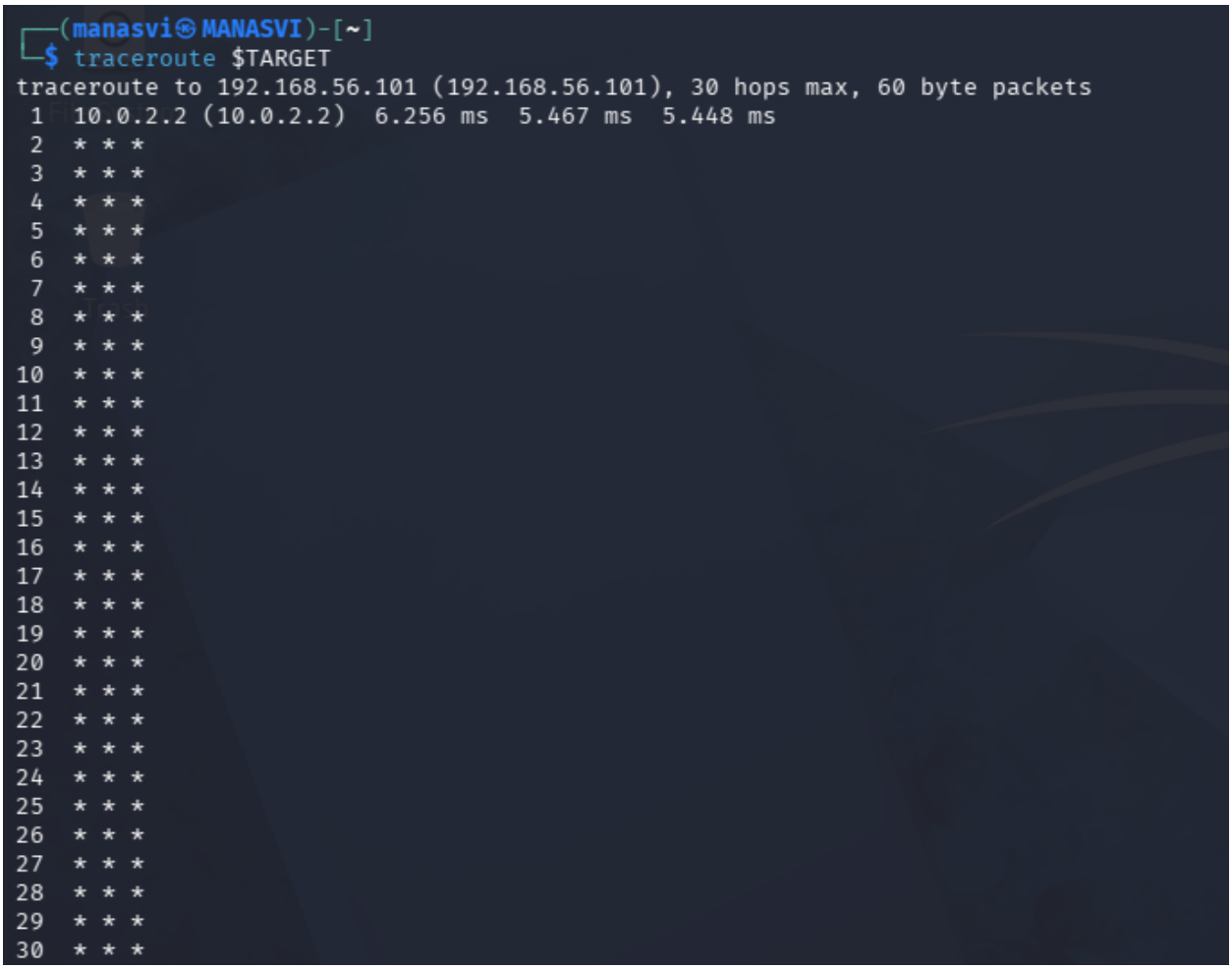
Ping is used to determine whether the target server is reachable through the network using ICMP packets.

ping -c 4 $TARGET

3. Traceroute – Identify Network Path

Traceroute is used to identify the path and intermediate network devices between the Kali machine and the target server.

traceroute $TARGET



4. IP Configuration

The ip a command displays the IP addresses, network interfaces, and network configuration of the Kali Linux system.

ip a

```
┌──(manasvi㉿MANASVI)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:81:53:59 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
       valid_lft 86022sec preferred_lft 86022sec
    inet6 fd17:625c:f037:2:ce76:d616:4f97:bf6d/64 scope global temporary dynamic
       valid_lft 86208sec preferred_lft 14208sec
    inet6 fd17:625c:f037:2:a00:27ff:fe81:5359/64 scope global dynamic mngtmpaddr noprefixroute
       valid_lft 86208sec preferred_lft 14208sec
    inet6 fe80::a00:27ff:fe81:5359/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

5. Network Connections

The ss command is used to display active network connections and listening TCP/UDP ports on the Kali system.

ss -tulnp

```
┌──(manasvi㉿MANASVI)-[~]
└─$ ss -tulnp
Netid      State      Recv-Q      Send-Q           Local Address:Port           Peer Address:Port     Process
```

6. Nmap Port Scanning

Nmap is used to identify open, closed, and filtered ports on the target server.

sudo nmap $TARGET

```
┌──(manasvi㉿MANASVI)-[~]
└─$ sudo nmap $TARGET
[sudo] password for manasvi:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-18 14:20 +0530
Nmap scan report for 192.168.56.101
Host is up (0.0046s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT     STATE SERVICE
5060/tcp open  sip
5061/tcp open  sip-tls

Nmap done: 1 IP address (1 host up) scanned in 6.68 seconds
```

7. Nmap Service Version Detection

Service version detection identifies the services running on open ports and determines their software versions.

sudo nmap -sV $TARGET

```
┌──(manasvi㉿MANASVI)-[~]
└─$ sudo nmap -sV $TARGET
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-18 14:22 +0530
Nmap scan report for 192.168.56.101
Host is up (0.0095s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT     STATE SERVICE  VERSION
5060/tcp open  sip?
5061/tcp open  sip-tls?

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 74.68 seconds
```

8. WhatWeb Technology Identification

WhatWeb is used to identify the web server, frameworks, CMS, programming languages, and other technologies used by the target website.

whatweb http://$TARGET

```
┌──(manasvi㉿MANASVI)-[~]
└─$ whatweb http://$TARGET
ERROR Opening: http://192.168.56.101 - execution expired
```

9. Curl HTTP Header Analysis

Curl is used to examine HTTP response headers and identify information that may reveal server or technology details.

curl -I http://$TARGET

```
┌──(manasvi㉿MANASVI)-[~]
└─$ curl -I http://$TARGET
curl: (7) Failed to connect to 192.168.56.101 port 80 after 21014 ms: Could not connect to server
```

10. Wget Server Response Analysis

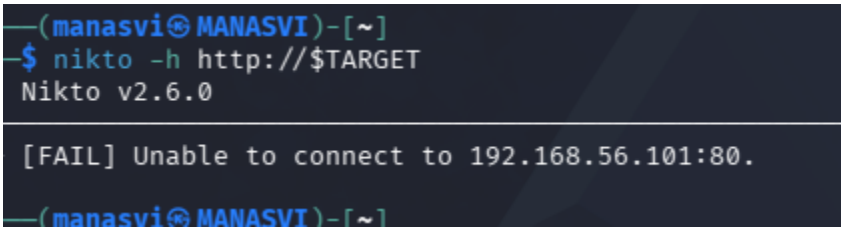
Wget is used to display the HTTP server response and headers without downloading the webpage.

wget --server-response --spider http://$TARGET

11. Nikto Vulnerability Scanning

Nikto is used to scan the web server for known vulnerabilities, misconfigurations, dangerous files, and information disclosure.

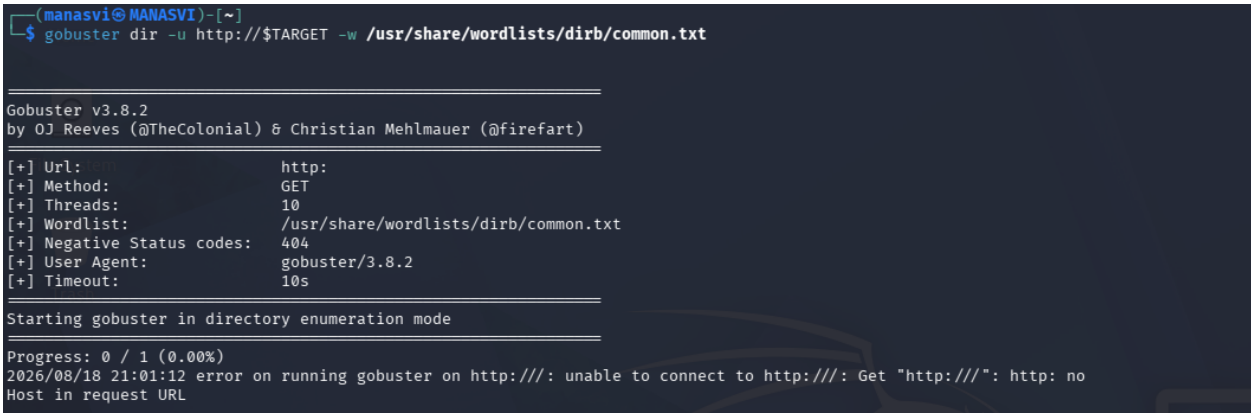
nikto -h http://$TARGET



12. Gobuster Directory Enumeration

Gobuster is used to discover hidden directories and files that may contain administrative pages, backup files, or other sensitive resources.

gobuster dir -u http://$TARGET -w /usr/share/wordlists/dirb/common.txt



Question 2: Web Application Vulnerability Assessment

A cybersecurity analyst is assigned an authorized vulnerable web application in an isolated laboratory environment for security testing. You are required to identify the application's technologies and attack surface using appropriate reconnaissance techniques, examine input fields, parameters, cookies, sessions, and HTTP requests, and use suitable tools to identify vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), insecure authentication, session-management weaknesses, directory traversal, and security misconfigurations. Use tools such as Burp Suite, OWASP ZAP, Nikto, Nmap, Curl, and browser developer tools to analyze the

application. Test the identified vulnerabilities only within the authorized laboratory environment, document the evidence and potential impact of each finding, classify the vulnerabilities according to severity, and recommend appropriate remediation and secure coding practices. Submit the commands/tool configurations, screenshots, observations, vulnerability findings, severity ratings, impacts, and recommendations.

Aim

To perform a security assessment of an authorized vulnerable web application using reconnaissance and web-security tools to identify vulnerabilities such as **SQL Injection, XSS, insecure authentication, session-management weaknesses, directory traversal, and security misconfigurations**, and to recommend suitable remediation measures.

Tools Used

- Kali Linux
- Nmap
- WhatWeb
- Curl
- Nikto
- Burp Suite
- OWASP ZAP
- Browser Developer Tools

Target

Use the IP address/URL of the vulnerable web application provided by your lab.

Procedure

1. Nmap – Port Scanning

Nmap is used to identify open, closed, and filtered ports on the target server.

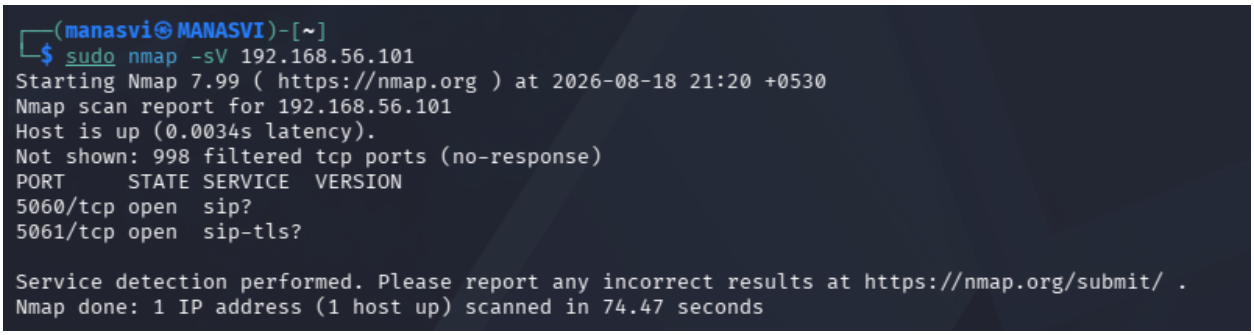
sudo nmap 192.168.56.101

Observation:
The scan identifies the ports and services accessible on the target.

2. Nmap – Service Version Detection

Nmap service detection identifies the services running on open ports and their software versions.

sudo nmap -sV 192.168.56.101

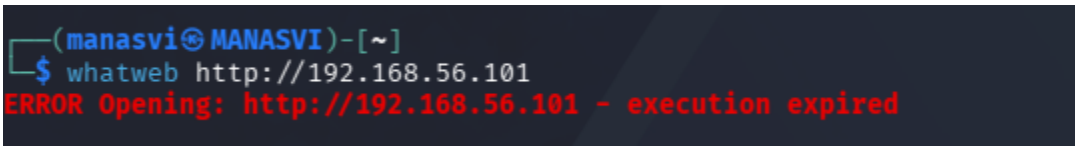


Observation:
The running services and their versions are recorded for further vulnerability analysis.

3. WhatWeb – Technology Identification

WhatWeb is used to identify the web server, frameworks, CMS, programming languages, and other technologies used by the application.

whatweb http://192.168.56.101

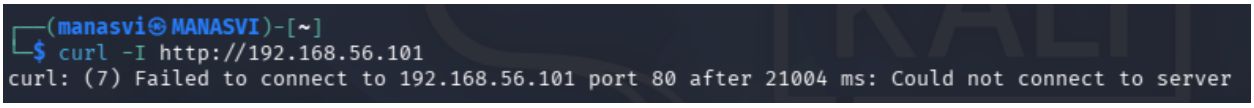


Observation:
The web technologies are identified if an accessible HTTP service is available.

4. Curl – HTTP Header Analysis

Curl is used to examine HTTP response headers and identify possible information disclosure.

curl -I http://192.168.56.101

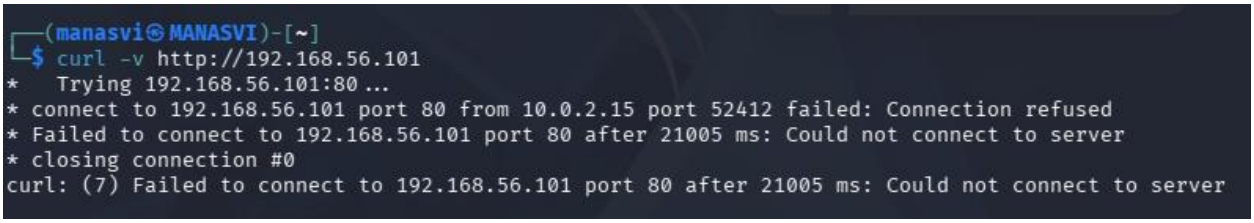


Observation:
Server information, cookies, redirects, and security headers are examined. If the HTTP port is filtered, no response headers will be obtained.

5. Curl – Detailed HTTP Analysis

Curl can also display detailed HTTP communication between the client and server.

curl -v http://192.168.56.101

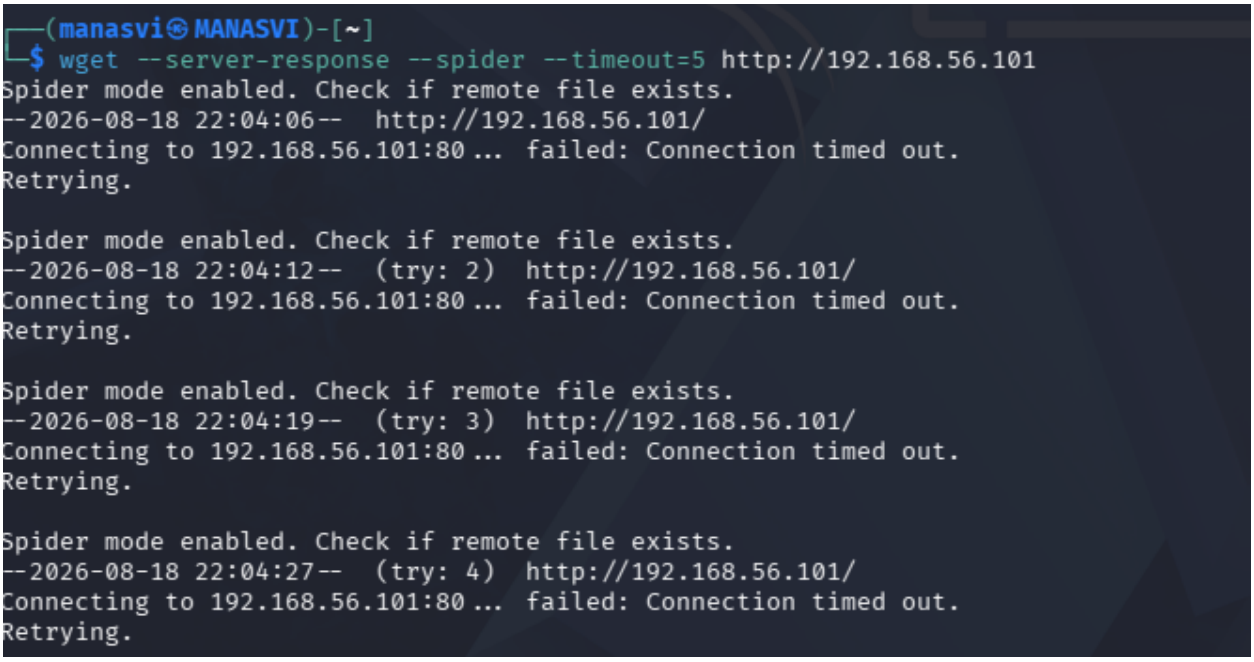


Observation:
The HTTP request, response, connection status, and server response information are analyzed.

6. Wget – Server Response Analysis

Wget is used to examine the HTTP server response without downloading the webpage.

wget --server-response --spider --timeout=5 http://192.168.56.101



Observation:
The HTTP response and headers are displayed if the web service is accessible.

7. Nikto – Web Vulnerability Scanning

Nikto is used to identify common web-server vulnerabilities, dangerous files, outdated components, and configuration weaknesses.

nikto -h http://192.168.56.101

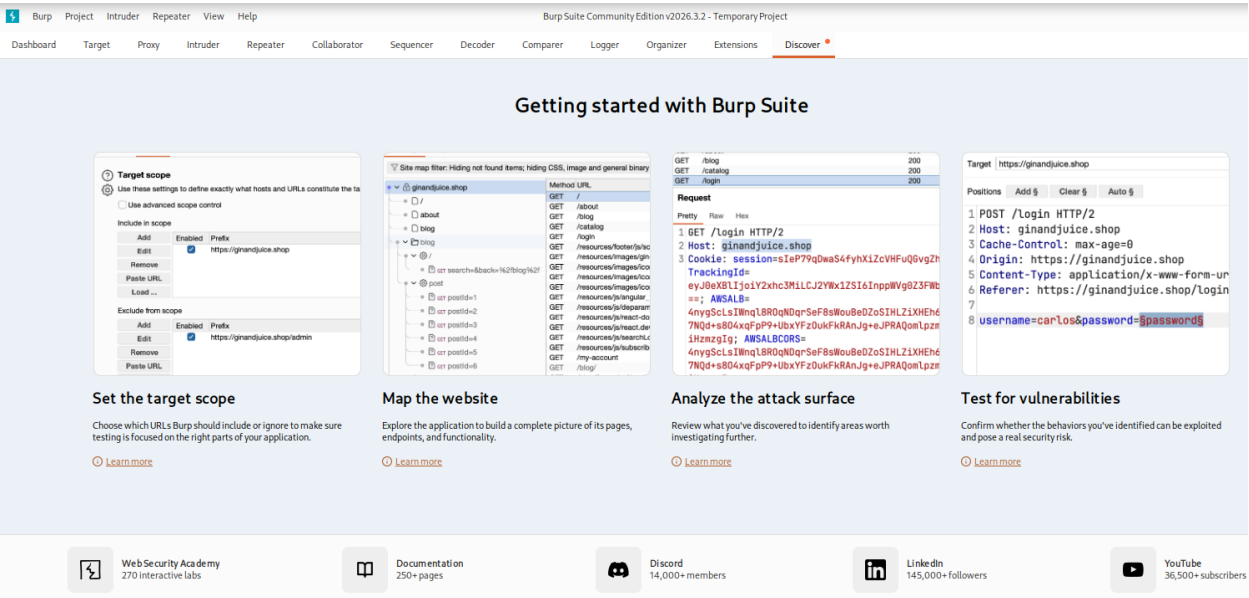
```
┌──(manasvi㉿MANASVI)-[~]
└─$ nikto -h http://192.168.56.101
- Nikto v2.6.0
---------------------------------------------------------------------------
+ [FAIL] Unable to connect to 192.168.56.101:80.
```

Observation:
Potential vulnerabilities and web-server misconfigurations are recorded.

8. Burp Suite – HTTP Request Analysis

Burp Suite is used to intercept and analyze HTTP requests and responses.

Burpsuite



The browser proxy is configured as:

127.0.0.1:8080

The application requests are examined for:

- Input parameters
- Login requests
- Cookies
- Session identifiers

- HTTP methods
- Server responses

Observation:
The application's request and response behavior is analyzed to identify security weaknesses.

9. OWASP ZAP – Web Application Scanning

OWASP ZAP is used to identify common web application vulnerabilities and security misconfigurations.

zaproxy

The authorized application is added as the target in ZAP.

Observation:
Potential vulnerabilities such as XSS, missing security headers, information disclosure, and other weaknesses are identified.

10. SQL Injection Testing

SQL Injection testing is performed on an authorized input field or parameter using a basic test input:

'

Observation:
Unexpected database errors or unusual application behavior may indicate improper input validation.

Impact:
SQL Injection may allow unauthorized access to or manipulation of application data.

Recommendation:
Use parameterized queries, prepared statements, proper input validation, and least-privilege database accounts.

11. Cross-Site Scripting (XSS) Testing

An authorized input field can be tested using:

<script>alert(1)</script>

Observation:
If the application executes the script instead of displaying it as text, the input may be vulnerable to XSS.

Impact:
XSS can allow malicious JavaScript to execute in a user's browser.

Recommendation:
Use proper output encoding, input validation, and an appropriate Content Security Policy.

12. Authentication and Session Management

The application's login mechanism and session cookies are examined using Burp Suite and browser Developer Tools.

The following cookie security attributes are checked:

Secure

HttpOnly

SameSite

Observation:
Weak authentication controls or insecure session cookies may allow unauthorized access or session-related attacks.

Recommendation:
Use strong authentication, HTTPS, secure cookie attributes, session expiration, session-ID regeneration, and rate limiting.

13. Directory Traversal Testing

File-related parameters are examined for improper path handling using a basic authorized test:

../

Observation:
If the application provides access outside the intended directory, a directory-traversal vulnerability may exist.

Impact:
Sensitive files outside the application's intended directory could potentially be accessed.

Recommendation:
Validate file paths, use allowlists, normalize paths, and avoid directly using user-controlled input in filesystem operations.

Vulnerability Classification

Vulnerability	Severity	Potential Impact	
SQL Injection	High/Critical	Unauthorized database access	
XSS	Medium/High	Malicious script execution	
Weak Authentication	High	Unauthorized account access	
Session Management Weakness	Medium/High	Session compromise	
Directory Traversal	High	Unauthorized file access	
Information Disclosure	Low/Medium	Exposure of system information	
Security Misconfiguration	Low/Medium	Increased attack surface	

Result

The authorized web application was assessed using Nmap, WhatWeb, Curl, Wget, Nikto, Burp Suite, OWASP ZAP, and browser Developer Tools. The application's technologies, attack surface, HTTP communication, input fields, authentication, and

session mechanisms were analyzed. Identified security weaknesses were classified according to their severity and impact, and appropriate remediation measures were recommended

Question 3: Security Header and HTTPS Configuration Assessment

A cybersecurity analyst is assigned an authorized web application hosted in an isolated laboratory environment to assess its HTTPS and security-header configuration. You are required to use Curl to examine HTTP response headers and Nmap SSL scripts to analyze the server's HTTPS/TLS configuration. Identify missing or improperly configured security headers such as Content-Security-Policy (CSP), HSTS, X-Frame-Options, and X-Content-Type-Options, and verify whether HTTP requests are automatically redirected to HTTPS. Examine the TLS configuration for weak protocols, insecure cipher suites, and certificate- related issues. Analyze the security purpose and potential impact of each misconfiguration,classify the findings according to their severity, and recommend appropriate HTTPS, TLS,certificate, and security-header hardening measures. Submit the commands used, screenshots,observations, identified risks, severity classification, and remediation recommendations.

Aim

To assess the **HTTPS, TLS, and security-header configuration** of an authorized web application using Curl and Nmap, identify security misconfigurations, analyze their impact, and recommend appropriate security hardening measures.

Theory

HTTPS and security-header assessment is performed to determine whether a web application securely protects communication between the user and server. HTTP security headers such as **Content-Security-Policy, HSTS, X-Frame-Options, and X-Content-Type-Options** provide additional protection against common web attacks. TLS configuration is also examined to identify weak protocols, insecure cipher suites, and certificate-related issues.

Tools Used

- Kali Linux
- Curl
- Nmap
- OpenSSL

1. HTTP Response Header Analysis

Curl is used to examine the HTTP response headers of the web application.

curl -I http://192.168.56.101

Observation:
The response headers are examined for security headers such as CSP, HSTS, X-Frame-Options, and X-Content-Type-Options.

---

2. HTTPS Response Header Analysis

Curl is used to examine the security headers provided over HTTPS.

curl -I https://192.168.56.101

Observation:
The HTTPS response is checked for security headers and other server information.

---

3. Check HTTP to HTTPS Redirection

This command checks whether HTTP requests are automatically redirected to HTTPS.

curl -I http://192.168.56.101

A secure redirection may appear as:

HTTP/1.1 301

Location: https://192.168.56.101/

Observation:
The Location header is checked to determine whether HTTP traffic is redirected to HTTPS.

---

4. Nmap SSL/TLS Cipher Analysis

Nmap is used to identify the supported TLS protocols and cipher suites.

sudo nmap -p 443 --script ssl-enum-ciphers 192.168.56.101

Observation:
Supported TLS versions and cipher suites are examined for weak or outdated configurations.

---

5. Nmap SSL Certificate Analysis

Nmap is used to obtain information about the server's SSL/TLS certificate.

sudo nmap -p 443 --script ssl-cert 192.168.56.101

Observation:
The certificate subject, issuer, validity period, and other certificate information are examined.

---

6. OpenSSL TLS Analysis

OpenSSL is used to inspect the TLS connection and certificate information.

openssl s_client -connect 192.168.56.101:443

Observation:
The negotiated TLS version, cipher, certificate, and certificate chain are ex

Security Header Analysis

Header	Purpose	Risk if Missing
Content-Security-Policy	Controls allowed content and scripts	Increased XSS risk
Strict-Transport-Security	Forces HTTPS communication	HTTPS downgrade risk
X-Frame-Options	Prevents unauthorized framing	Clickjacking risk
X-Content-Type-Options	Prevents MIME sniffing	Content-type attacks

Severity Classification

Finding	Severity	Impact
Weak TLS protocol	High	Weakens encrypted communication
Insecure cipher suite	High/Medium	Reduces communication security
Invalid/expired certificate	Medium/High	Reduces server identity trust
Missing HSTS	Medium	Allows possible HTTP downgrade
Missing CSP	Medium	Increases XSS risk
Missing X-Frame-Options	Low/Medium	Increases clickjacking risk
Missing X-Content-Type-Options	Low	Allows MIME-sniffing risks

Result

The HTTPS, TLS, and security-header configuration of the authorized web application was assessed using **Curl, Nmap SSL scripts, and OpenSSL**. Security headers, HTTPS redirection, TLS protocols, cipher suites, and certificate configuration were examined, and appropriate security hardening measures were recommended.

Question 4: Broken Access-Control Assessment

A cybersecurity analyst is assigned an authorized student portal in an isolated laboratory environment containing separate student and administrator test accounts. Using OWASP ZAP, capture and analyze requests generated while accessing student records and identify test object identifiers in URLs, forms, cookies, or API requests. Using only the test identifiers provided by the faculty, determine whether one student account can access another student's records or whether a normal user can directly access administrator-only resources. Compare the server responses for authorized and unauthorized requests, explain the difference between authentication and authorization failures,

classify the identified access-control weaknesses based on their severity and impact, and recommend appropriate server-side authorization checks, role validation, and secure object-reference mechanisms. Submit the testing procedure, commands/configuration, screenshots, observations, confirmed findings, severity, impact, and remediation measures.

Aim

To assess the access-control mechanisms of an authorized student portal using **OWASP ZAP** and determine whether student and administrator resources are properly protected through server-side authorization checks.

Theory

Access control determines **what an authenticated user is allowed to access**. In this assessment, separate student and administrator test accounts are used to examine whether users can access resources belonging to other users or administrator-only functions.

A major access-control weakness is **Broken Access Control**, where the server fails to properly verify whether the authenticated user has permission to access a requested resource. This can occur when applications rely only on object IDs in URLs or client-side restrictions instead of performing authorization checks on the server.

Tools Used

- Kali Linux
- OWASP ZAP
- Web Browser
- Browser Developer Tools

1. Start OWASP ZAP

OWASP ZAP is used to intercept and analyze requests between the browser and the authorized student portal.

zaproxy

Configure the browser to use the ZAP proxy:

127.0.0.1:8080

Observation:
ZAP captures HTTP requests and responses generated while accessing the student portal.

---

2. Login Using Student Account

Log in to the portal using the **faculty-provided student test account**.

Navigate to the student's authorized record.

In ZAP, inspect the captured request.

For example, a request may contain:

/student?id=101

or:

/api/student/101

Observation:
The object identifier used to access the student's record is recorded.

---

3. Test Authorized Object Access

First, capture the request for the student's own record and note the test identifier provided by the faculty.

For example:

/student?id=101

The server response is recorded as the **authorized response**.

Observation:
The student's own record is successfully returned because the account has permission to access it.

---

4. Test Another Faculty-Provided Student Identifier

Using **only the test identifier provided by the faculty**, modify the object identifier in the intercepted request.

For example:

/student?id=102

Send the modified request through ZAP.

Observation:
The response is compared with the authorized request.

A secure application should reject the request with an appropriate response such as:

HTTP/1.1 403 Forbidden

or another appropriate authorization response.

If the server returns another student's information with a successful response, this may indicate a **Broken Object Level Authorization (BOLA/IDOR)** vulnerability.

---

5. Test Administrator Resource Access

Using the normal student account, identify an administrator-only URL from the **faculty-provided test environment**.

For example:

/admin

Request the resource while still authenticated as the student.

Observation:
A properly configured application should deny access to administrator-only resources.

If the student account receives administrator content successfully, this indicates an **authorization failure**.

---

6. Compare Server Responses

The responses for authorized and unauthorized requests are compared based on:

- HTTP status code
- Response body
- Redirect behavior
- Returned data
- Error messages
- User role information

Example:

Request	Expected Result
Student → Own Record	Access permitted
Student → Another Student's Record	Access denied
Student → Administrator Resource	Access denied
Administrator → Authorized Admin Resource	Access permitted

Authentication vs Authorization

Authentication determines **who the user is**.

Example:

The student successfully logs into the portal using valid credentials.

Authorization determines **what that authenticated user is allowed to access**.

Example:

A student is authenticated but should not be allowed to access another student's records or administrator-only pages.

Therefore, a user can be **successfully authenticated but still fail authorization**.

Findings and Severity

Finding	Severity	Impact
Student can access another student's record	High	Unauthorized disclosure of student information
Student can access administrator resources	Critical/High	Unauthorized administrative access
Missing server-side authorization checks	High	Users may access restricted resources
Predictable object identifiers alone	Medium/High	May facilitate unauthorized object access
Finding	**Severity**	**Impact**

The final severity should be based on the **actual evidence obtained from the authorized laboratory application**.

Recommendations

1. Implement **server-side authorization checks** for every protected request.
2. Verify the authenticated user's identity before returning an object.
3. Enforce role-based access control for student and administrator functions.
4. Do not rely only on hidden buttons or client-side restrictions.
5. Use secure object-reference mechanisms and avoid exposing predictable identifiers where appropriate.
6. Apply least-privilege access to student and administrator resources.
7. Return appropriate authorization errors such as HTTP 403 Forbidden.

8. Log and monitor unauthorized access attempts.
9. Perform authorization checks on every API endpoint, not only on the user interface.
10. Conduct regular access-control testing.

Result

The authorized student portal was assessed using **OWASP ZAP** by capturing and analyzing requests from separate student and administrator test accounts. Test object identifiers were examined and authorized and unauthorized requests were compared. Any confirmed access-control weaknesses were classified according to their severity and impact, and appropriate server-side authorization and role-validation measures were recommended.

Question 5: Automated Scanning and Manual Validation

A cybersecurity analyst is assigned an authorized web application hosted in a controlled cybersecurity laboratory for vulnerability assessment. Use Nikto to identify web-server misconfigurations and outdated components, and use Wapiti or OWASP ZAP to perform an automated web-application security scan. Compare the vulnerabilities reported by the selected tools and categorize them into misconfiguration, information disclosure, authentication, and injection issues. Select at least three findings for safe manual validation within the authorized laboratory environment and determine whether each finding is confirmed, a false positive, or requires further investigation. Assign an appropriate severity level based on likelihood and potential impact, and recommend practical remediation measures for all confirmed vulnerabilities. Submit a comparative report containing tool outputs, screenshots, validation evidence, severity classification, remediation measures, and overall conclusions.

Aim

To perform an automated vulnerability assessment of an authorized web application using **Nikto and OWASP ZAP**, compare the identified vulnerabilities, manually validate selected findings, classify their severity, and recommend appropriate remediation measures.

Theory

Automated vulnerability scanning helps identify common weaknesses in web servers and applications. **Nikto** is mainly used to identify web-server misconfigurations, outdated components, dangerous files, and information disclosure. **OWASP ZAP** is used to identify web-application vulnerabilities such as injection issues, security misconfigurations, authentication weaknesses, and other common security problems.

Automated scanners may produce **false positives**, so important findings should be manually validated in the authorized laboratory environment before they are considered confirmed vulnerabilities.

Tools Used

- Kali Linux
- Nikto
- OWASP ZAP
- Web Browser

Target

Target IP: 192.168.56.101

---

1. Nikto Web Server Scanning

Nikto is used to identify web-server vulnerabilities, outdated components, dangerous files, and configuration issues.

nikto -h http://192.168.56.101

Observation:
The Nikto output is examined for outdated software, missing security headers, information disclosure, and other web-server weaknesses.

---

2. OWASP ZAP Scanning

OWASP ZAP is used to perform automated web-application security testing.

Start ZAP:

zaproxy

The authorized application is added as the target in ZAP.

The scan can identify issues such as:

- Security misconfigurations
- Information disclosure
- Injection-related issues
- Authentication and session weaknesses
- Missing security headers

Observation:
The vulnerabilities reported by ZAP are recorded for comparison with the Nikto results.

---

3. Compare the Scan Results

The results from Nikto and ZAP are compared and categorized.

Category	Example Finding	Source
Misconfiguration	Missing security headers	Nikto/ZAP
Information Disclosure	Server version exposed	Nikto/ZAP
Authentication	Weak authentication configuration	ZAP

Category	Example Finding	Source
Injection	Possible XSS/injection	ZAP

The same vulnerability reported by both tools provides stronger evidence, while findings reported by only one tool require further validation.

---

4. Manual Validation – Security Header

A reported missing security header can be manually checked using Curl:

curl -I http://192.168.56.101

Check for headers such as:

Content-Security-Policy

Strict-Transport-Security

X-Frame-Options

X-Content-Type-Options

Observation:
If a security header reported as missing by the scanner is actually absent from the response, the finding can be considered **confirmed**.

---

5. Manual Validation – Information Disclosure

Check the server response:

curl -v http://192.168.56.101

Look for information such as:

Server:

X-Powered-By:

Observation:
If unnecessary server or technology information is exposed, the information-disclosure finding is confirmed.

---

6. Manual Validation – Web Application Input

For an authorized input field, submit a harmless test value and observe the response.

Example:

'

For XSS testing in the authorized laboratory:

<script>alert(1)</script>

Observation:
If the application safely handles the input, the automated finding may be a **false positive**. If unsafe behavior is observed, the finding requires further analysis and may be confirmed.

---

7. Finding Classification

Each automated finding is classified as:

Confirmed

The vulnerability reported by the scanner is reproduced during manual validation.

False Positive

The scanner reports a vulnerability, but manual testing shows that the vulnerability does not actually exist.

Requires Further Investigation

There is insufficient evidence to confirm or reject the finding safely.

---

8. Severity Classification

Severity	Description
Critical	Could result in complete compromise or highly sensitive data exposure
High	Could result in significant unauthorized access or data disclosure
Medium	Could provide limited unauthorized access or increase attack risk
Low	Minor security weakness with limited direct impact
Informational	Security-related information without a direct exploitable impact

---

9. Comparative Findings

Finding	Nikto	ZAP	Validation	Severity
Missing security header	✓	✓	Confirmed/False Positive	Low/Medium
Server information disclosure	✓	✓	Confirmed/False Positive	Low
Possible XSS	—	✓	Confirmed/False Positive	Medium/High
Authentication weakness	—	✓	Requires investigation	High
Outdated component	✓	✓/—	Confirmed/Investigation	Medium/High

The final status and severity should be based on the **actual results obtained from the laboratory target**.

Recommendations

1. Update outdated web-server software and components.
2. Remove unnecessary information from HTTP responses.
3. Configure appropriate security headers.
4. Implement strong authentication and session-management controls.

5. Validate and encode user input to prevent injection and XSS.
6. Remove unnecessary files and directories from the web server.
7. Apply secure configuration and least-privilege principles.
8. Manually verify important automated findings before remediation.
9. Regularly perform vulnerability assessments after updates.
10. Maintain proper security logs and monitoring.

Result

The authorized web application was assessed using **Nikto and OWASP ZAP**. The findings from both tools were compared and categorized into misconfiguration, information disclosure, authentication, and injection issues. Selected findings were manually validated and classified as **confirmed, false positive, or requiring further investigation**. Appropriate severity levels and remediation measures were assigned based on the potential impact of each finding.